Pet Catalog Filter System

System overview

The pet catalog's filter system lets adopters narrow the public/pets listing by six independent criteria: species, breed, size, age range, shelter (manual multi-select), and location (geolocation or postal code, resolved to nearby shelters). This document explains how each piece of code works, and separately traces the complete data flow for each filter, from a user's click through to the rendered result.

High-Level Architecture

Data flows through four layers, in this order:

- Backend endpoints (pets.controller.js / pets.service.js, shelters.controller.js / shelters.service.js) - validate requests and query the database
- **petsApi.ts**: the frontend's typed API client, translating between frontend filter state and backend query parameters
- **PetCatalog.tsx**: owns all filter state, runs every TanStack Query call, and merges filter inputs into the final request sent to GET /pets
- **PetFilterBar.tsx** and **ShelterLocationFilter.tsx**: presentational components that render the filter UI and report changes back up via callback props

> *The core architectural principle throughout: state lives in exactly one place (PetCatalog.tsx), and every component below it is a relay - it displays what it's given and reports changes upward, rather than maintaining its own competing copy of the truth.*

Two Kinds of Filtering

Species, breed, size, and age range are LIVE filters - every change immediately updates state and triggers a refetch, with no confirmation step. Location search is different: it requires an explicit "Find Nearby Shelters" trigger, because it involves a two-step network chain that would be wasteful to re-run on every incidental interaction. This hybrid pattern matches how production location-search UIs are commonly built (e.g. Airbnb, Zillow). Category filters refine live, location search is a deliberate action.

1. Backend - pets.service.js

This file builds the Prisma query for GET /pets, given a set of already-validated filters from the controller.

1.1 toArray and matchFilter - Normalizing Filter Inputs

Query parameters can arrive from Express as three different shapes: undefined (not provided), a single string (provided once), or an array (provided multiple times, e.g. ?species=1&species=2). toArray() collapses all three into one guaranteed shape - always an array - so every filter downstream can assume a consistent input.

```
const toArray = (value) => {
  if (value === undefined) return [];
  return Array.isArray(value) ? value : [value];
};
```

matchFilter() then decides how to shape the actual Prisma condition, based on how many values are present:

```
const matchFilter = (values) =>
  values.length === 1 ? values[0] : { in: values };
```

A single value becomes a plain equality match; multiple values become Prisma's `in` operator. Both are functionally correct, the single-value case just produces slightly cleaner SQL.

1.2 buildAgeFilter - Converting Months to Date Boundaries

Pets store petDOB, not a static age, so age is always computed live rather than going stale. minAge/maxAge (in months) are converted into a petDOB date-range comparison before querying:

```
const monthsAgo = (months) => {
  const today = new Date();
  return new Date(today.getFullYear(), today.getMonth() - months, today.getDate());
};

const buildAgeFilter = (minAge, maxAge) => {
  const dobFilter = {};
  if (minAge !== undefined) {
    dobFilter.lte = monthsAgo(Number(minAge));
  }
  if (maxAge !== undefined) {
    const today = new Date();
    const cutoff = new Date(
      today.getFullYear(),
      today.getMonth() - (Number(maxAge) + 1),
      today.getDate(),
    );
    dobFilter.gt = cutoff; // strict greater-than, not gte
  }
  return Object.keys(dobFilter).length > 0 ? dobFilter : undefined;
};
```

- **Why minAge Is Straightforward**

  "At least N months old" means the pet's DOB must fall on or before the date N months ago - older pets have earlier birth dates, so the comparison is petDOB <= monthsAgo(N). Verified correct against every realistic DOB/age combination.

- **Why maxAge Needs the +1 Month and Strict gt**

  A naive implementation would mirror minAge exactly: dobFilter.gte = monthsAgo(maxAge). This looks correct but is wrong at the boundary. The bug was caught by testing maxAge=35 against a real pet (DOB 2023-07-10) whose displayed age correctly showed 35 months - yet the naive filter excluded that pet.

  The root cause: the function that computes a pet's displayed age rounds DOWN if today's day-of-month hasn't yet reached the DOB's day-of-month:

  ```
  if (today.getDate() < dobDate.getDate()) months -= 1;
  ```

  This means "35 months," as displayed, can correspond to a DOB almost a full month earlier than a naive calculation would assume. The naive maxAge cutoff and the displayed age were quietly using two different definitions of "35 months."

  The fix shifts the cutoff back one additional month AND uses a strict greater-than comparison instead of gte. Both changes are required together. This combination was verified by sweeping every day-of-month against a wide range of ages, confirming zero mismatches.

  > *This filtering logic is entirely separate from how petAge is formatted in the response. The filter operates only on dates; it never touches the formatted petAge string - the two are decoupled.*

1.3 Promise.all - Parallel Data and Count Queries

Pagination requires two pieces of information one query can't provide together: the actual page of results, and the total count across ALL matching records. findMany with skip/take never sees the rows it skips - the total has to be requested separately.

```
const [pets, total] = await Promise.all([
  prisma.pet.findMany({ where, skip, take: limit, select: {...} }),
  prisma.pet.count({ where }),
]);
```

Since these two queries are independent, Promise.all runs them concurrently rather than sequentially. The total wait is whichever query takes longer, not the sum of both.

1.4 The Response Transformation

Raw Prisma results are reshaped before being sent to the client - three distinct kinds of transformation in the same .map() call:

- Pass-through: petID, petName, petPhoto are copied unchanged
- Restructure: breed.species.speciesName (nested three levels deep in the raw Prisma shape) is flattened to breed.speciesName
- Compute: petDOB becomes petAge, a formatted string computed live at request time - this value doesn't exist as a stored column

> *Sending the raw Prisma shape directly would couple the API's contract to the database's internal structure, and require the frontend to duplicate age-calculation logic. Computing petAge once, server-side, keeps exactly one source of truth.*

2. Backend - pets.controller.js

The controller reads raw HTTP query parameters, validates them, and only calls the service with clean, correctly-typed values - rejecting the request early if anything is wrong.

2.1 Two Validation Categories

Closed Sets - Validated Upfront

page, limit, minAge, maxAge, and size all have an objective definition of "valid" checkable without touching the database. These get checked immediately, with a 400 on failure.

Open Sets - Left to the Database

breed has no fixed list to check against - the valid list lives in the BREED table and can grow over time. A nonsense value like "Book" isn't rejected; the query simply matches zero rows. Validating against an open, growing set would require either a second database round-trip, or a hardcoded list that inevitably goes stale.

2.2 Species - Migrated from Open to Closed

Species was originally filtered by NAME (?species=Dog), the same open-set pattern as breed - but /breeds already filtered by speciesID, forcing the frontend to maintain two parallel arrays (speciesIDs for the /breeds cascade, speciesNames for /pets) purely to satisfy two different backend conventions for one user selection.

The fix switched /pets to filter by speciesID, matching /breeds. Being numeric, it also became strictly validated:

```
const speciesValues = toArray(species).map((raw) => Number(raw));
if (speciesValues.some((s) => !Number.isInteger(s))) {
  return next(badRequest("species must be an array of integers (speciesID)"));
}
```

> *This removed the parallel-array duplication entirely - toggleSpecies no longer tracks speciesNames; names are looked up on demand at render time when needed for pills.*

2.3 shelterID - A Deliberate Exception

shelterID is numeric, like species now is, but was deliberately kept on a more permissive pattern:

```
// shelterID is not validated per spec - a non-numeric value is coerced to an
// ID that can never match, so the query naturally returns 0 results instead of
erroring.
const shelterIDValues = toArray(shelterIDRaw).map((raw) => {
  const parsed = Number(raw);
```

```
    return Number.isInteger(parsed) ? parsed : -1;
});
```

Rather than 400-ing on an invalid shelterID, this coerces it to -1 - an ID guaranteed to never exist - so the query proceeds normally but matches nothing for that value, without failing the whole request if other valid IDs are also present.

This was a discussed, deliberate tradeoff: strict rejection would surface bugs earlier, but since shelterID always comes from the app's own /shelters endpoint, the realistic risk of malformed input is low. The graceful pattern was kept here specifically, while species moved to strict validation.

3. Backend - PostGIS Geolocation (shelters.service.js)

Prisma doesn't support PostGIS's geography functions natively, so GET /shelters/nearby uses raw SQL via prisma.$queryRaw for the distance/radius portion.

3.1 Building a Geographic Point

```
ST_SetSRID(ST_MakePoint(${lng}, ${lat}), 4326)
```

Two distinct steps, worth keeping separate:

- ST_MakePoint(lng, lat) - constructs a raw coordinate pair. Note the order: longitude first, then latitude - a common source of bugs if reversed.
- ST_SetSRID(..., 4326) - tags the point with SRID 4326, the standard reference system for GPS lat/lng. Not a calculation - it tells PostGIS which coordinate system's rules apply.

3.2 ST_Distance vs. ST_DWithin - Measuring vs. Filtering

ST_Distance - Measures

```
ST_Distance("shelterLocation", ST_SetSRID(ST_MakePoint(${lng}, ${lat}), 4326)) / 1000
AS distance
```

Calculates the great-circle distance between the shelter's stored location and the search point, in meters, divided by 1000 for kilometers.

ST_DWithin - Filters

```
AND ST_DWithin("shelterLocation", ST_SetSRID(ST_MakePoint(${lng}, ${lat}), 4326),
${radius * 1000})
```

Not a calculated number - a boolean WHERE condition. Used instead of comparing a manual ST_Distance because PostGIS can use spatial indexes to cheaply rule out far-away points, without the full distance calculation on every row.

3.3 Rounding the Response Distance

```
distance: Math.round(Number(shelter.distance) * 100) / 100,
```

Number(shelter.distance) - raw SQL results aren't automatically typed the way Prisma's query builder is, so this ensures a genuine number.

Math.round(x * 100) / 100 - rounds to 2 decimal places by shifting the decimal point before and after rounding. Used. instead of .toFixed(2), which returns a string, keeping distance a genuine number in the response.

4. Backend – Isolated Geocoding Module

GET /shelters/nearby accepts an optional postalCode parameter as an alternative to lat/lng, resolved via the zipcodes npm package. Country-specific logic is deliberately isolated so supporting a different country later requires changing only one small module.

4.1 Why Isolate Country-Specific Logic

If the controller imported zipcodes directly, swapping countries later would mean hunting down every call site. Instead, everything outside the geocoding module calls one generic function:

```
resolveCoordsFromPostalCode(postalCode: string): { lat: number, lng: number } | null
```

Changing countries later means writing one new implementation file and updating one import line - nothing else in the codebase changes.

4.2 File Structure

```
server/src/services/geocoding/
  index.js                    <- the ONLY thing anything else imports
  usPostalCodeGeocoder.js      <- the only file allowed to mention "zipcodes" or "US"

// index.js
const { resolveCoordsFromPostalCode } = require("./usPostalCodeGeocoder");
module.exports = { resolveCoordsFromPostalCode };

// usPostalCodeGeocoder.js
const zipcodes = require("zipcodes");

const resolveCoordsFromPostalCode = (postalCode) => {
  const result = zipcodes.lookup(postalCode);
  if (!result) return null;
  return { lat: result.latitude, lng: result.longitude };
};
```

4.3 Two Paths, One Interface

Geolocation and postal code both ultimately produce the same shape: a lat and a lng number. Everything downstream treats them identically, with no awareness of which method produced them.

- **Geolocation**: the browser's own location hardware hands back coordinates directly - never touches the geocoding module.
- **Postal code**: the backend resolves it via resolveCoordsFromPostalCode before running the same distance query used for direct lat/lng.

Validation order: postal code is checked first if provided; otherwise falls back to lat/lng validation; the request is rejected only if neither was given.

5. Frontend - TanStack Query Concepts

Every data-fetching call in this feature uses TanStack Query's useQuery hook. This section explains the mechanics that recur throughout the frontend files, so the file-specific sections that follow can reference them rather than re-explain them each time.

5.1 queryKey, queryFn, and the Caching Decision

Every useQuery call needs a queryKey (uniquely identifying "what is this request") and a queryFn (the function that actually fetches data). Before ever calling queryFn, TanStack Query checks its internal cache against the queryKey: if fresh cached data already exists for that exact key, it's returned immediately with no network request; otherwise queryFn runs.

> *This caching decision happens automatically inside useQuery - nowhere in this codebase is there manual "check the cache first" logic. Providing an accurate queryKey is the developer's entire responsibility; TanStack Query handles the rest.*

Crucially, queryKey comparison is structural (based on content), not based on object reference - two differently-referenced objects with identical contents are treated as the same key. This distinction is the root cause of a real bug covered in Section 8.6.

5.2 enabled - Gating Queries on Prerequisites

enabled is a boolean (or expression) that gates whether a query is allowed to run at all - a separate, earlier check than the caching decision above. If enabled is false, TanStack Query doesn't check the cache or call queryFn; the query simply stays idle.

Used throughout this feature: the breeds query only runs once a species is selected; the shelters-nearby query only runs once a location search has been triggered; the pets query waits until any in-flight nearby-shelter search has settled.

5.3 keepPreviousData - Avoiding UI Flicker

By default, when a query's key changes, its data resets to undefined while the new fetch is in flight - triggering a loading state. keepPreviousData changes this: the previous result stays visible while the new one loads in the background, avoiding a jarring skeleton-flash on every minor filter change or page click.

5.4 refetchOnWindowFocus - A Default Worth Knowing

TanStack Query, by default, automatically re-validates active queries whenever the browser tab regains focus. This surfaced during debugging: a UI update that should have happened automatically (but didn't, due to the bug in Section 8.6) appeared to "fix itself" after switching tabs and back - not because the underlying bug resolved, but because this unrelated default behavior forced a fresh evaluation that happened to route around it.

6. Frontend - petsApi.ts

The typed API client layer, sitting between the frontend's filter state and the backend endpoints. Its interfaces fall into two distinct categories, worth keeping separate.

6.1 Request Shapes vs. Response Shapes

Species, Breed, Shelter, NearbyShelter, and PetCard describe data coming IN from the API - shapes the backend sends back. PetFilters describes something this frontend constructs and sends OUT - the exact shape GET /pets' query parameters expect.

```
export interface PetFilters {
  speciesID?: number[];
  breedName?: string[];
  size?: string[];
  minAge?: string;
  maxAge?: string;
  shelterID?: number[];
}
```

Every field is optional (the ? marker) for the same reason req.query fields are optional on the backend: a user might not have set that particular filter, and the field should be omitted from the request entirely rather than sent as an empty value.

6.2 Building the Request - the ?.length Pattern

```
if (filters.speciesID?.length) params.species = filters.speciesID;
```

This single line does two checks in sequence: ?. (optional chaining) protects against filters.speciesID being undefined at all - without it, reading .length on undefined would throw. .length being truthy then separately confirms the array actually has content, not just that it exists - distinguishing "never touched this filter" (undefined) from "explicitly selected zero items" (empty array), both of which should result in the param being omitted.

6.3 The Species Name-to-ID Migration

PetFilters.speciesID was originally speciesName: string[], matching the backend's original name-based filtering. When /pets moved to filter by speciesID (Section 3.2), this interface's field was both renamed and re-typed, and getPets' param-building logic updated to match - a coordinated change across the backend controller, this file, and every frontend component that constructs a PetFilters object.

7. Frontend - PetCatalog.tsx (State Ownership)

This component owns every piece of filter state and runs every query. Its Filters interface is deliberately broader than petsApi.ts's PetFilters, because it tracks things other queries need internally, not just what eventually gets sent to GET /pets.

7.1 Why Filters Is Broader Than PetFilters

```
export interface Filters {
  speciesIDs: number[];
  breedNames: string[];
  size: string[];
  minAge: string;
  maxAge: string;
  selectedShelterIDs: number[];
  nearbyShelterIDs: number[];
}
```

speciesIDs exists here specifically because the breeds query needs it (GET /breeds filters by ID) - a need that has nothing to do with what eventually gets sent to /pets. Filters represents everything this page needs to remember; PetFilters represents only what one specific outgoing request needs. The translation between them happens explicitly when petFilters is constructed.

7.2 Two Shelter Arrays - selectedShelterIDs and nearbyShelterIDs

Manually-picked shelters (via the Shelter dropdown) and shelters found via a location search are tracked as two separate arrays, combined via union only when building the final request:

```
const mergedShelterIDs = [...new Set([...filters.selectedShelterIDs,
...filters.nearbyShelterIDs])];
```

These are kept separate rather than merged into one array immediately because they have fundamentally different lifecycles: selectedShelterIDs updates instantly on a checkbox click; nearbyShelterIDs only updates after a full async search round-trip completes. Keeping them separate also lets the UI distinguish (visually, via badges) which shelters came from which source, and lets a location search be cleared without disturbing manual selections, and vice versa.

7.3 The seq Trick - Forcing a Query to Re-run

```
interface NearbySearchParams {
  lat?: number;
  lng?: number;
  postalCode?: string;
  radius: number;
  seq: number;
}
```

Recall from Section 6.1: queryKey comparison is structural, not reference-based. If a user clicks "Find Nearby Shelters" twice in a row with identical lat/lng/radius, the resulting nearbySearch object would be structurally identical both times - and

TanStack Query would treat the second click as "the same query I already ran," skipping it entirely, even though the user explicitly clicked the button again expecting a fresh search.

```
const handleFindNearby = (location, radius) => {
  setNearbySearch((prev) => ({ ...location, radius, seq: (prev?.seq ?? 0) + 1 }));
};
```

seq increments on every call, guaranteeing the object can never be structurally identical to its previous value - forcing TanStack Query to always treat a button click as a genuinely new query, regardless of whether the underlying search parameters happen to repeat.

7.4 useEffect vs. queryFn - Separating Fetch From Side Effects

```
useEffect(() => {
  if (isNearbySuccess && nearbyShelterSResult.length > 0) {
    updateFilters({ nearbyShelterIDs: nearbyShelterSResult.map((s) => s.shelterID) });
  }
  // An empty result is intentionally not committed - nearbyShelterIDs stays as-is
}, [nearbyShelterSResult, isNearbySuccess]);
```

queryFn's only job should be to fetch and return data - not to also reach into unrelated state as a side effect. This logic lives in a useEffect instead, watching for the shelters-nearby query's result changing, and reacting to it separately. This keeps fetching and state-updating as two distinct concerns, each easier to reason about independently.

> *An empty result is deliberately NOT committed to nearbyShelterIDs. This preserves any manually-selected shelters' pets from disappearing just because a separate, unrelated location search happened to find nothing.*

7.5 The Union-Merge and the [-1] Override

mergedShelterIDs.length > 0 ? mergedShelterIDs : undefined initially meant "a completed, zero-result location search" and "no location filter ever attempted" both produced the same outcome: no shelter constraint applied at all, silently reverting to an unfiltered catalog. This was a real, deliberately corrected gap:

```
shelterID:
  nearbySearchEmpty && filters.selectedShelterIDs.length === 0
    ? [-1]
    : mergedShelterIDs.length > 0
      ? mergedShelterIDs
      : undefined,
```

When a location search genuinely completes with zero results, and no shelter is separately, manually selected, this forces the same -1 sentinel pattern used for invalid shelterID values on the backend (Section 3.3) - guaranteeing the pets query correctly returns nothing, rather than silently falling back to "no filter at all." The condition specifically checks selectedShelterIDs (not nearbyShelterIDs) so that a manually-selected shelter's pets keep showing even if a separate location search fails.

7.6 Case Study - The queryKey Staleness Bug

Symptom: after a zero-result location search, clicking "reset the location filter" appeared to do nothing - the catalog stayed empty until the browser tab was switched away and back, at which point the correct results reappeared.

Diagnosis Process

- Confirmed the reset function was actually being called (console log fired on click)
- Confirmed the pets query's enabled condition was true throughout, ruling out a gating issue
- Confirmed calling refetch() manually immediately fixed it - proving the underlying logic and data were correct, and the query itself was capable of producing the right result
- The fact that only an external trigger (refetch(), or a tab-focus event) fixed it - never an automatic re-evaluation - pointed at a queryKey detection issue, not a logic bug

Root Cause

```
queryKey: ["pets", page, filters],  // the bug
```

The pets query was keyed on filters, but the [-1] override in Section 8.5 depends on nearbySearchEmpty - a value computed outside of filters entirely, from the separate shelters-nearby query's own state. In the reset scenario, filters itself (specifically nearbyShelterIDs, already empty before and after) didn't structurally change - but petFilters.shelterID did change (from [-1] to undefined), because nearbySearchEmpty changed. Since queryKey never included that change, TanStack Query never detected anything different and never refetched automatically.

Fix

```
queryKey: ["pets", page, petFilters],  // key on what queryFn actually consumes
```

> *General principle: a queryKey must reflect everything the queryFn actually depends on - not just the state that conceptually feels like the input. Whenever a query's behavior is influenced by something computed BETWEEN raw state and the actual fetch call, that derived value - not the upstream state - is usually the safer thing to key on.*

8. Frontend - PetFilterBar.tsx

Renders the species, breed, size, and age filter controls. Presentational - receives filters and callback props from PetCatalog.tsx, owns no filter state of its own (only local UI state, like whether the panel is expanded).

8.1 The Presentational Component Pattern

filters arrives as a prop, not local useState - PetFilterBar displays selections and reports changes to them, it does not own or store them. When a checkbox is clicked, the component calls updateFilters(...) (received as a prop), which runs in PetCatalog.tsx and updates the real state there; PetFilterBar then simply re-renders with the new filters prop it's handed.

8.2 toggleSpecies - Why React Requires New Arrays

```
const toggleSpecies = (speciesID) => {
  const speciesIDs = filters.speciesIDs.includes(speciesID)
    ? filters.speciesIDs.filter((id) => id !== speciesID)
    : [...filters.speciesIDs, speciesID];
  updateFilters({ speciesIDs, breedNames: [] });
};
```

React decides whether to re-render by checking if a new state value is a DIFFERENT OBJECT from the old one (reference equality) - not by inspecting its contents. Mutating an array in place (e.g. .push()) would leave the same object in memory with different contents; React would see "same reference" and skip the re-render entirely, silently breaking the UI. .filter() and the spread operator (...) both build a genuinely new array every time, which is why they're used here instead of any in-place mutation.

breedNames is reset to [] on every species change - a breed selection tied to a now-changed species set no longer makes sense, matching the cascading-dropdown UX pattern.

8.3 Case Study - The agePillLabel Bug

The original nested-ternary implementation:

```
const agePillLabel = filters.minAge
  ? filters.maxAge
    ? `${filters.minAge}-${filters.maxAge} mo`
    : `${filters.minAge}+ mo`
  : `Up to ${filters.maxAge} mo`;
```

There are four real combinations of minAge/maxAge being set or empty, but this structure only has three distinct branches - "neither set" silently falls into the same branch as "only maxAge set," producing "Up to mo" (a broken, empty-looking label) when neither field actually has a value.

This specific broken case never reached the screen - the pill's render condition already guards against it (filters.minAge !== "" || filters.maxAge !== "") - but it illustrates a general risk: a nested ternary that LOOKS like it handles every case cleanly can still have fewer branches than the number of real input combinations, silently sharing a branch between two cases that shouldn't produce the same result.

Fixed version, with an explicit fourth branch:

```
const agePillLabel =
  filters.minAge && filters.maxAge
    ? `${filters.minAge}-${filters.maxAge} mo`
    : filters.minAge
      ? `${filters.minAge}+ mo`
      : filters.maxAge
        ? `Up to ${filters.maxAge} mo`
        : "";
```

8.4 Type Assertions and Component Reusability

```
onToggle={(value) => toggleSpecies(value as number)}
```

CheckboxDropdown is one reusable component serving species (numeric IDs), breed, and size (both strings) - so its FilterOption.value type is necessarily generic (string | number), since TypeScript has no way to know, from the component's own definition, which specific case applies at any given call site. as number is a type ASSERTION, not a conversion - it performs no runtime check or transformation; it simply tells TypeScript "trust me, this is a number here," based on knowledge the developer has (this dropdown's options were built with numeric values) that TypeScript's generic type can't infer on its own.

9. Frontend - ShelterLocationFilter.tsx

Renders the Shelter multi-select and the Distance/location search panel - the most complex component in this feature, coordinating five distinct pieces of local state alongside the props it receives.

9.1 Draft vs. Committed State

pendingRadius (what's currently selected in the radio buttons) and searchedRadius (what radius the last COMPLETED search actually used) are deliberately separate. Consider: a user searches at 50km, then - without searching again - clicks the 10km radio button just to look. The location pill must keep showing "50 km" (what actually produced the current results), not "10 km" (what's merely selected but never searched). The pill prefers searchedRadius, falling back to pendingRadius only if no search has completed yet:

```
`Nearby: ${searchedRadius ?? pendingRadius} km`
```

9.2 The Geolocation Browser API

navigator.geolocation.getCurrentPosition() takes two separate FUNCTIONS as arguments - a success callback and a failure callback - rather than returning a Promise. This is an older-style API, predating the Promise-based conventions used everywhere else in this app (useQuery, async/await).

```
navigator.geolocation.getCurrentPosition(
  (position) => { /* success: browser hands back a position object with real coords */
},
  () => { /* failure: browser calls this instead, with no coords */ },
);
```

The browser itself decides, internally, which of the two functions to call - this is not conditional logic the developer writes; it's the API's own contract. On success, coords gets populated; on failure, geoError is set and coords stays null.

9.3 Priority Resolution - Location vs. Postal Code

Both input methods stay always-enabled in the UI (never disabled based on the other), but only one is actually used per search:

```
const handleFindNearby = () => {
  if (coords) {
    runSearch(coords);
  } else if (/^\d{5}$/.test(zipInput)) {
    runSearch({ zip: zipInput });
  }
};
```

A successfully-obtained current location always takes priority over a typed zip code, even if both are present. "Given" specifically means coords is populated (a successful geolocation result) - not merely that the button was clicked. If geolocation fails, coords stays null, meaning it's treated as not supplied at all, and a valid zip code in the field can still be used.

10. Design Decisions Log

11.1 Live vs. Staged Filtering

Species, breed, size, and age refine results instantly. Location search requires an explicit trigger. This split was deliberately modeled on real-world precedent - Airbnb, Zillow, and similar location-aware apps universally separate "cheap, exploratory refinement" (instant) from "the primary, more expensive location search" (explicit action), because a location search typically involves a genuinely different cost (a permissions prompt, a network round-trip to resolve a location, then a second query) that shouldn't fire on every incidental interaction.

11.2 The Shelter Visual-Distinction Feature

Initially deferred, then built: shelters found via a successful location search show as checked in the Shelter dropdown, but visually distinct (a gold accent and "Nearby" badge) from manually-picked shelters. This required extending CheckboxDropdown to accept a second, separately-styled set of checked values (nearbyValues) alongside the original selectedValues, rather than merging both sources into one indistinguishable checked list.

11.3 Postal Code Scope - Zip-Only, Not Full Address

Full address geocoding (via a service like Google's Geocoding API) was considered and deliberately scoped down to US zip codes only, resolved via a bundled, offline npm package (zipcodes) rather than a live third-party API call. This avoided a new external dependency, API key management, and network-based failure modes, while still meeting the actual need - shelters already store a zip code, and zip-level precision is sufficient for "find shelters near me."

11. End-to-End Flows, Per Filter

Each subsection traces one filter's complete path - from the user's interaction through every file involved, to the rendered result. Intended as a standalone reference: read just one subsection to understand that filter's full behavior, without needing to piece it together from the file-by-file sections above.

11.1 Species Filter

1. User checks a species option in PetFilterBar's Species CheckboxDropdown.
2. toggleSpecies rebuilds speciesIDs (add or remove the clicked ID).
3. updateFilters (PetCatalog.tsx) updates filters state and resets page to 1.
4. petFilters.speciesID is rebuilt from the new speciesIDs.
5. The pets query's queryKey changes → GET /pets fires.
6. pets.controller.js validates speciesID as an array of integers.
7. pets.service.js applies matchFilter to build the Prisma where clause.
8. Response returns; data.map() transforms it into the client-facing shape.
9. Pet cards re-render with the filtered results.

> *Species also independently feeds the Breed filter: speciesIDs changing also changes the breeds query's queryKey, and - since enabled: filters.speciesIDs.length > 0 - triggers GET /breeds, repopulating the Breed dropdown's options.*

11.2 Breed Filter

1. User checks a breed option (disabled until a species is selected).
2. toggleBreed rebuilds breedNames.
3. updateFilters updates filters state.
4. petFilters.breedName is rebuilt.
5. GET /pets fires, with the open-set validation from Section 3.1 - no upfront rejection, nonsense values simply match zero rows.
6. Response returns; pet cards re-render.

11.3 Size Filter

The simplest flow - no cascading, no backend transformation beyond the enum check:

1. User checks a size option.
2. toggleSize rebuilds size.
3. updateFilters updates filters state.
4. petFilters.size is rebuilt.
5. GET /pets fires, validated against the closed VALID_SIZES enum in the controller.
6. Response returns; pet cards re-render.

11.4 Age Range Filter

1. User types into the minAge or maxAge input.
2. filters.minAge / filters.maxAge updates.
3. isAgeRangeInvalid (a client-side guard) recalculates - if min exceeds max, the pets query's enabled condition is blocked and an inline warning shows.
4. Once valid, petFilters.minAge / petFilters.maxAge is rebuilt.
5. GET /pets fires.

6. pets.controller.js validates both as integers and re-checks min <= max server-side.
7. pets.service.js's buildAgeFilter converts months to the asymmetric date-boundary comparison (Section 2.2).
8. Response includes each pet's petAge as a formatted string.
9. Pet cards render the age pill.

11.5 Shelter Filter (Manual Multi-Select)

1. User checks a shelter option in the Shelter CheckboxDropdown.
2. toggleShelter rebuilds selectedShelterIDs.
3. onSelectedShelterIDsChange relays up to PetCatalog.tsx's updateFilters.
4. mergedShelterIDs recomputes - a union with any nearbyShelterIDs already present.
5. petFilters.shelterID is rebuilt.
6. GET /pets fires, using the graceful -1 coercion pattern for any malformed value (Section 3.3).
7. Response returns; pet cards re-render.

11.6 Location Filter (Geolocation / Postal Code)

The most complex flow, with two possible entry points converging on one path:

- **Geolocation**: icon click → handleGeolocateClick → browser's getCurrentPosition → success populates coords, or failure sets geoError (Section 10.2)
- **Postal code**: zip typed into the input field, validated client-side as 5 digits before the search button enables

Once "Find Nearby Shelters" Is Clicked

1. handleFindNearby resolves priority - coords wins if present (Section 10.3).
2. runSearch sets searchedRadius and searchMethod, then calls onFindNearby.
3. PetCatalog.tsx's handleFindNearby sets nearbySearch, with the seq increment (Section 8.3).
4. The Distance dropdown closes.
5. GET /shelters/nearby fires - with EITHER lat/lng directly OR a postalCode resolved server-side via the isolated geocoding module (Section 5).
6. shelters.controller.js validates the request.
7. shelters.service.js runs the PostGIS ST_DWithin / ST_Distance query (Section 4).
8. Response returns.

Handling the Response

1. If non-empty: a useEffect commits the shelter IDs to nearbyShelterIDs (Section 8.4), which render as visually-distinct badges in the Shelter dropdown (Section 11.2).
2. If empty: nearbyShelterIDs is deliberately left uncommitted, and nearbySearchEmpty becomes true.
3. If no shelter is also manually selected, the [-1] override (Section 8.5) forces the pets query to correctly show zero results, with a message and a reset link in place of the pet grid.

Resetting the Location Filter

Via the Reset Filter button, the location pill's X, or the empty-state's reset link, all wired to the same complete reset function (Section 10.4). This clears nearbySearch, nearbyShelterIDs, and every local input, causing nearbySearchEmpty to correctly return to false and the pets query to refetch without the location constraint.